

Κεφάλαιο 5

Μονάδες Προγράμματος & Διαδικασίες στη Fortran

FUNCTION, SUBROUTINE, ορίσματα, internal procedures και MODULE

Στόχος: να μπορείς να χωρίζεις ένα πρόγραμμα σε καθαρές, επαναχρησιμοποιήσιμες διαδικασίες και να επιλέγεις σωστά μεταξύ FUNCTION, SUBROUTINE και MODULE.

5.1 Γιατί χωρίζουμε ένα πρόγραμμα

Ένα μεγάλο πρόγραμμα είναι δύσκολο να διαβαστεί, να ελεγχθεί και να διορθωθεί. Χωρίζουμε μια εργασία σε μικρές διαδικασίες όταν ένα κομμάτι κώδικα έχει σαφή, ανεξάρτητο σκοπό: για παράδειγμα «υπολόγισε άθροισμα τετραγώνων», «μέτρησε τιμές πάνω από όριο» ή «εκτύπωσε αναφορά».

Μια καλή διαδικασία έχει όνομα που περιγράφει τι κάνει, δέχεται μόνο τα δεδομένα που χρειάζεται και επιστρέφει καθαρά το αποτέλεσμα. Έτσι το κυρίως πρόγραμμα περιγράφει τη ροή της λύσης, ενώ η διαδικασία κρατά τις λεπτομέρειες του υπολογισμού.

5.2 Μονάδες προγράμματος

Στη Fortran το κυρίως πρόγραμμα, μια FUNCTION, μια SUBROUTINE και ένα MODULE είναι μονάδες προγράμματος. Κάθε μονάδα έχει δικό της χώρο δηλώσεων και, ως ασφαλής συνήθεια, γράφουμε `implicit none` σε κάθε ανεξάρτητη μονάδα.

```
program main
implicit none
! δηλώσεις και εκτελέσιμες προτάσεις
contains
! εδώ τοποθετούνται οι εσωτερικές διαδικασίες
end program main
```

Το `contains` χωρίζει το εκτελέσιμο μέρος του κυρίως προγράμματος από τις εσωτερικές διαδικασίες. Ό,τι βρίσκεται μετά το `contains` δεν εκτελείται απευθείας από πάνω προς τα κάτω: εκτελείται μόνο όταν καλέσουμε τη διαδικασία του.

5.3 FUNCTION - όταν θέλουμε μία τιμή

Μια FUNCTION μοιάζει με μαθηματική συνάρτηση: δέχεται δεδομένα και επιστρέφει μία τιμή. Τη χρησιμοποιούμε μέσα σε έκφραση, σε εκχώρηση ή σε PRINT/WRITE. Δεν τη καλούμε με CALL.

```
program demo_function
implicit none
real :: values(4), result

values = [1.0, 2.0, 3.0, 4.0]
result = square_sum(values)
print '(a,f8.2)', 'Άθροισμα τετραγώνων = ', result

contains

real function square_sum(x)
real, intent(in) :: x(:)
square_sum = sum(x**2)
end function square_sum

end program demo_function
```

Τι γίνεται εδώ:

Η `square_sum(values)` είναι κλήση FUNCTION και επιστρέφει μία REAL τιμή. Η γραμμή `real function square_sum(x)` δηλώνει ότι η συνάρτηση επιστρέφει REAL. Η εκχώρηση `square_sum = ...` είναι υποχρεωτική: δίνει την τελική τιμή της FUNCTION. Για `values=[1,2,3,4]`, εμφανίζεται 30.00.

Το `x(:)` δηλώνει έναν μονοδιάστατο πίνακα `assumed-shape`: η διαδικασία δέχεται πίνακα οποιουδήποτε μήκους. Το `intent(in)` σημαίνει ότι η διαδικασία επιτρέπεται να διαβάσει το `x` αλλά όχι να αλλάξει τα στοιχεία του.

5.4 SUBROUTINE - όταν εκτελούμε μια εργασία

Μια SUBROUTINE καλείται αποκλειστικά με CALL. Είναι κατάλληλη όταν θέλουμε να εκτελέσουμε μια ενέργεια ή να επιστρέψουμε περισσότερα από ένα αποτελέσματα μέσω ορισμάτων.

```
program demo_subroutine
implicit none
real :: values(4), total, average

values = [1.0, 2.0, 3.0, 4.0]
call statistics(values, total, average)
print '(a,f8.2)', 'Άθροισμα = ', total
print '(a,f8.2)', 'Μέσος όρος = ', average

contains

subroutine statistics(x, s, mean)
real, intent(in) :: x(:)
real, intent(out) :: s, mean

s = sum(x)
mean = s / real(size(x))
end subroutine statistics

end program demo_subroutine
```

Η CALL statistics(values,total,average) δεν είναι έκφραση. Ζητά από τη SUBROUTINE να κάνει την εργασία της. Το intent(out) για τα s και mean δηλώνει ότι αυτά είναι αποτελέσματα που θα γραφτούν από τη διαδικασία.

Κανόνας επιλογής: FUNCTION όταν το νόημα είναι «υπολόγισε και δώσε μου μία τιμή». SUBROUTINE όταν το νόημα είναι «κάνε μια εργασία» ή όταν χρειαζόμαστε πολλά αποτελέσματα.

5.5 Ορίσματα, εικονικές μεταβλητές και INTENT

Στην κλήση `call statistics(values,total,average)`, τα `values`, `total` και `average` του κυρίως προγράμματος λέγονται πραγματικά ορίσματα. Τα `x`, `s` και `mean` μέσα στη διαδικασία λέγονται εικονικά ορίσματα. Η αντιστοίχιση γίνεται από τη θέση τους: πρώτο με πρώτο, δεύτερο με δεύτερο κ.ο.κ.

Οι πιο σημαντικές δηλώσεις πρόθεσης είναι:

`intent(in)`: η διαδικασία μόνο διαβάζει το όρισμα. `intent(out)`: η διαδικασία γράφει το όρισμα ως αποτέλεσμα. `intent(inout)`: η διαδικασία διαβάζει την αρχική τιμή και στη συνέχεια την αλλάζει.

```
program change_example
implicit none
real :: temperature

temperature = 20.0
call add_offset(temperature, 2.5)
print *, temperature

contains

subroutine add_offset(x, offset)
real, intent(inout) :: x
real, intent(in) :: offset
x = x + offset
end subroutine add_offset

end program change_example
```

Εδώ εμφανίζεται 22.5. Το `temperature` αλλάζει επειδή αντιστοιχεί στο `x`, το οποίο είναι `intent(inout)`. Το `offset` είναι μόνο είσοδος και δεν αλλάζει.

5.6 Εσωτερικές διαδικασίες

Οι διαδικασίες που γράφουμε μετά το `contains` του κυρίως προγράμματος λέγονται εσωτερικές (internal procedures). Είναι η καλύτερη αρχή για το πρώτο μας κεφάλαιο διαδικασιών, επειδή βρίσκονται δίπλα στο πρόγραμμα που τις χρησιμοποιεί και η Fortran γνωρίζει ήδη τη διεπαφή τους.

Μια εσωτερική διαδικασία μπορεί να χρησιμοποιεί ονόματα του περιβάλλοντος που την περιέχει, όμως ως καθαρή πρακτική προτιμάμε να της περνάμε ρητά τα δεδομένα που χρειάζεται μέσω ορισμάτων. Έτσι είναι πιο εύκολο να την ξαναχρησιμοποιήσουμε και να τη δοκιμάσουμε.

5.7 Εξωτερικές διαδικασίες

Μια εξωτερική διαδικασία γράφεται έξω από το κυρίως πρόγραμμα. Μπορεί να βρίσκεται στο ίδιο ή σε διαφορετικό αρχείο. Ο καθηγητής παρουσιάζει αυτή τη μορφή, αλλά για αρχάριους απαιτεί περισσότερη προσοχή: η κάθε μονάδα χρειάζεται τις δικές της δηλώσεις και το κυρίως πρόγραμμα χρειάζεται να γνωρίζει τη FUNCTION.

```
program use_external_function
implicit none
real, external :: cube
real :: x

x = 2.0
print *, cube(x)
end program use_external_function

real function cube(x)
implicit none
real :: x
cube = x**3
end function cube
```

Τι είναι το `cube` μέσα στη FUNCTION: στη γραμμή `real function cube(x)`, το `real` δηλώνει ότι η FUNCTION επιστρέφει REAL και το `cube` γίνεται αυτόματα το όνομα της μεταβλητής αποτελέσματος. Άρα η γραμμή `cube = x**3` αποθηκεύει την τιμή που θα επιστρέψει η FUNCTION. Δεν χρειάζεται δεύτερη δήλωση `real :: cube`, γιατί αυτή περιέχεται ήδη στην επικεφαλίδα της FUNCTION. Αντίθετα, το `x` είναι όρισμα εισόδου και δηλώνεται ρητά με `real :: x`.

Η δήλωση `real, external :: cube` ενημερώνει το κυρίως πρόγραμμα ότι το `cube` είναι εξωτερική FUNCTION που επιστρέφει REAL. Στη συνέχεια του κεφαλαίου θα δούμε ότι τα MODULE προσφέρουν πιο οργανωμένη και ασφαλή επαναχρησιμοποίηση.

5.8 MODULE και module procedures

Ένα MODULE είναι άρθρωμα: ένας οργανωμένος χώρος όπου τοποθετούμε σταθερές, τύπους δεδομένων και διαδικασίες που ανήκουν μαζί. Για να χρησιμοποιήσει ένα πρόγραμμα τα περιεχόμενά του, γράφει use όνομα_module.

```
module vector_tools
  implicit none
  contains

  real function square_sum(x)
    real, intent(in) :: x(:)
    square_sum = sum(x**2)
  end function square_sum

end module vector_tools

program use_module
  use vector_tools, only: square_sum
  implicit none
  real :: v(3)

  v = [1.0, 2.0, 3.0]
  print *, square_sum(v)
end program use_module
```

Πολύ σημαντικό: όταν MODULE και κυρίως πρόγραμμα βρίσκονται στο ίδιο αρχείο, γράφουμε πρώτα ολόκληρο το MODULE και μετά το PROGRAM.

Γιατί γράφουμε only: square_sum εδώ; Σε αυτό το μικρό παράδειγμα το MODULE έχει μόνο τη square_sum, άρα θα λειτουργούσε ακριβώς το ίδιο και η απλούστερη γραμμή use vector_tools. Το only δεν είναι απαραίτητο ακόμη: το χρησιμοποιούμε από τώρα ως καλή συνήθεια, επειδή δηλώνει καθαρά ποια ονόματα επιλέγουμε να φέρουμε στο κυρίως πρόγραμμα και αποφεύγει μελλοντικές συγκρούσεις ονομάτων.

```
! αν θέλουμε όλα τα PUBLIC ονόματα του module
use vector_tools

! αν θέλουμε μόνο επιλεγμένα PUBLIC ονόματα
use vector_tools, only: square_sum

! αν αργότερα υπήρχε και η count_above
use vector_tools, only: square_sum, count_above
```

Το only: μπορεί να περιέχει όσα ονόματα χρειαζόμαστε, όπως functions, subroutines ή παραμέτρους, αρκεί να είναι διαθέσιμα ως public στο MODULE. Με use vector_tools χωρίς only εισάγονται όλα τα public ονόματα του module.

5.9 PUBLIC και PRIVATE

Το PUBLIC/PRIVATE ελέγχει ποια ονόματα ενός MODULE είναι διαθέσιμα σε κώδικα που το κάνει use. Μια απλή και ασφαλής οργάνωση είναι να κάνουμε όλα τα ονόματα private και να δηλώνουμε public μόνο όσα θέλουμε να χρησιμοποιεί το κυρίως πρόγραμμα.

```
module calculations
  implicit none
  private
  public :: square_sum
  contains

  real function square_sum(x)
    real, intent(in) :: x(:)
    square_sum = helper_value(x**2)
  end function square_sum

  real function helper_value(x)
    real, intent(in) :: x(:)
    helper_value = sum(x)
  end function helper_value

end module calculations

program demo_access
  use calculations, only: square_sum
  implicit none
  real :: v(3)

  v = [1.0, 2.0, 3.0]
  print *, square_sum(v) ! επιτρέπεται
  ! print *, helper_value(v) ! δεν επιτρέπεται: είναι private
end program demo_access
```

Στο παράδειγμα η square_sum είναι public, άρα το κυρίως πρόγραμμα μπορεί να τη χρησιμοποιήσει. Η helper_value δεν δηλώνεται public, άρα παραμένει private: μπορεί να χρησιμοποιηθεί μέσα στο MODULE, όπως από τη square_sum, αλλά δεν μπορεί να κληθεί από το κυρίως πρόγραμμα. Το x και στις δύο FUNCTION είναι τοπικό όρισμα της διαδικασίας και δεν ήταν ούτως ή άλλως διαθέσιμο έξω από αυτήν.

5.10 Διαδικασίες με συστοιχίες

Η σύνδεση με το Κεφάλαιο 4 είναι άμεση: ένας πίνακας περνιέται ως όρισμα ακριβώς όπως ένας απλός αριθμός. Για μονοδιάστατο πίνακα που μπορεί να έχει διαφορετικό μήκος κάθε φορά, η απλή σύγχρονη δήλωση είναι `x(:)`.

```
program count_values
implicit none
real :: a(6), limit
integer :: n_above

a = [1.0, 5.0, -2.0, 7.5, 3.0, 8.0]
limit = 4.0
call count_above(a, limit, n_above)
print *, 'Πλήθος πάνω από το όριο: ', n_above

contains

subroutine count_above(x, threshold, n)
real, intent(in) :: x(:), threshold
integer, intent(out):: n
integer :: i

n = 0
do i = 1, size(x)
if (x(i) > threshold) n = n + 1
end do
end subroutine count_above

end program count_values
```

Η διαδικασία δεν χρειάζεται ξεχωριστό `n` για το μέγεθος του `x`, επειδή η `size(x)` το βρίσκει. Η `n` του παραδείγματος είναι διαφορετική: είναι το αποτέλεσμα, δηλαδή το πλήθος των στοιχείων πάνω από το `threshold`. Με τα δεδομένα αυτά εμφανίζεται 3.

5.11 Δυναμικές και assumed-shape συστοιχίες σε διαδικασίες

Όταν το πλήθος στοιχείων γίνεται γνωστό κατά την εκτέλεση, χρησιμοποιούμε allocatable. Η λέξη allocate δεσμεύει χώρο για την επιθυμητή διάσταση και η deallocate τον ελευθερώνει. Στο πλαίσιο μιας διαδικασίας, ένα όρισμα x(:) είναι assumed-shape: η διαδικασία δέχεται το σχήμα του πραγματικού πίνακα που της δίνεται.

```
program dynamic_result
implicit none
real :: x(3)
real, allocatable :: y(:)

x = [1.0, 2.0, 3.0]
call doubled_copy(x, y)
print *, y
deallocate(y)

contains

subroutine doubled_copy(x, y)
real, intent(in) :: x(:)
real, allocatable, intent(out) :: y(:)

allocate(y(size(x)))
y = 2.0*x
end subroutine doubled_copy

end program dynamic_result
```

Το y είναι allocatable και έξω από τη διαδικασία και μέσα σε αυτή. Η SUBROUTINE δεσμεύει y με το ίδιο μέγεθος με το x και επιστρέφει το νέο διάνυσμα [2,4,6]. Μετά τη χρήση, το deallocate(y) αποδεσμεύει τη μνήμη.

Σημείο προσοχής: δεν χρησιμοποιούμε μια allocatable συστοιχία πριν γίνει allocate. Για τις πρώτες εφαρμογές, το x(:) μαζί με size(x) είναι η βασική μορφή που αρκεί για διαδικασίες πάνω σε arrays.

5.12 Εφαρμογή Lab 7 - από εκφώνηση σε σχέδιο

Το Lab 7 ζητά έναν πραγματικό πίνακα $A(N)$, το άθροισμα των τετραγώνων του με internal FUNCTION, την ίδια εργασία με internal SUBROUTINE και τέλος SUBROUTINE που επιστρέφει πλήθος στοιχείων μεγαλύτερων από όριο.

Πριν γράψουμε κώδικα, τον χωρίζουμε σε τέσσερις ευθύνες:

1. Κυρίως πρόγραμμα: ορίζει N , διαβάζει τα στοιχεία A , καλεί τις διαδικασίες και εκτυπώνει τα τελικά αποτελέσματα. 2. `fun_internal(A)`: επιστρέφει μία REAL τιμή, άρα είναι FUNCTION. 3. `sub_internal(A)`: εκτυπώνει ή επιστρέφει το ίδιο αποτέλεσμα, άρα είναι SUBROUTINE. 4. `count_above(A,limit,count)`: δέχεται δεδομένα και επιστρέφει ακέραιο πλήθος, άρα είναι SUBROUTINE με intent(out) για το count.

```
program lab7_plan
implicit none
integer, parameter :: n = 5
integer :: i, count
real :: a(n), limit, s

do i = 1, n
  read *, a(i)
end do

s = fun_internal(a)
call sub_internal(a)
read *, limit
call count_above(a, limit, count)

contains
! εδώ γράφουμε τις τρεις διαδικασίες
end program lab7_plan
```

Το παραπάνω είναι σκαλωσιά ανάλυσης, όχι η τελική λύση του εργαστηρίου. Μας βοηθά να επιλέξουμε πρώτα τις διαδικασίες και τα ορίσματά τους, πριν γεμίσουμε τις λεπτομέρειες των βρόχων και των υπολογισμών.

5.13 Από εκφώνηση σε διαδικασίες

Χρησιμοποίησε αυτή τη σειρά σκέψης: (1) Ποια εργασία επαναλαμβάνεται ή έχει ανεξάρτητο νόημα; (2) Χρειάζεται ένα μόνο αποτέλεσμα; Τότε σκέψου FUNCTION. (3) Χρειάζονται πολλά αποτελέσματα ή ενέργεια; Τότε σκέψου SUBROUTINE. (4) Ποια δεδομένα είναι είσοδοι, ποια έξοδοι και ποια αλλάζουν; Γράψε intent(in), intent(out), intent(inout). (5) Αν η διαδικασία αφορά ομάδα συναφών υπολογισμών, τοποθέτησέ την σε MODULE.

5.14 Συχνά λάθη

- Κλήση FUNCTION με CALL. Η FUNCTION πρέπει να χρησιμοποιείται σε έκφραση, π.χ. $s=f(x)$.
- Κλήση SUBROUTINE σαν να επιστρέφει τιμή. Η SUBROUTINE καλείται με call.
- Παράλειψη της εκχώρησης στο όνομα της FUNCTION. Χωρίς αυτήν δεν έχει αποτέλεσμα να επιστρέψει.
- Ασυμφωνία στη σειρά, στον τύπο ή στο πλήθος των ορισμάτων.
- Αλλαγή δεδομένου εισόδου χωρίς να το έχουμε προβλέψει. Το intent(in) προστατεύει από αυτό.
- Γραφή internal procedure πριν από το contains ή εκτελέσιμων εντολών μετά από contains.
- Χρήση allocatable πίνακα πριν από allocate ή χωρίς deallocate όταν τελειώσει η χρήση του.

5.15 Ασκήσεις εφαρμογής

1. Γράψε internal FUNCTION cube(x), η οποία δέχεται REAL x και επιστρέφει το x^3 . Κάλεσέ την από ένα κυρίως πρόγραμμα για $x=2.0$.
2. Γράψε internal SUBROUTINE rectangle(a,b,area,perimeter), η οποία δέχεται τις πλευρές ενός ορθογωνίου και επιστρέφει εμβαδό και περίμετρο.
3. Για πίνακα θερμοκρασιών t(6), γράψε FUNCTION mean_value(x) που επιστρέφει τον μέσο όρο. Χρησιμοποίησε size(x).
4. Γράψε SUBROUTINE count_negative(x,nneg) που επιστρέφει το πλήθος των αρνητικών στοιχείων ενός REAL πίνακα.
5. Μετέτρεψε την Άσκηση 4 σε MODULE vector_checks και χρησιμοποίησέ την με use vector_checks, only: count_negative.
6. Δήλωσε allocatable πίνακα y(:). Γράψε SUBROUTINE make_squares(x,y) που δεσμεύει το y στο σωστό μέγεθος και το γεμίζει με τα τετράγωνα του x.
7. Υλοποίησε την εργασία του Lab 7: πίνακας A(N), internal FUNCTION για άθροισμα τετραγώνων, internal SUBROUTINE για την ίδια αναφορά και SUBROUTINE που μετρά στοιχεία πάνω από όριο.

Στην ενότητα αυτή δίνονται μόνο εκφωνήσεις. Οι λύσεις θα δοθούν σε ξεχωριστό τεύχος.

5.16 Checklist Κεφαλαίου 5

- Μπορώ να εξηγήσω τη διαφορά FUNCTION και SUBROUTINE.
- Μπορώ να καλέσω FUNCTION με έκφραση και SUBROUTINE με CALL.
- Ξέρω τι είναι πραγματικό και εικονικό όρισμα.
- Μπορώ να χρησιμοποιήσω intent(in), intent(out) και intent(inout).
- Ξέρω τον ρόλο του contains και μπορώ να γράψω internal procedure.
- Μπορώ να χρησιμοποιήσω MODULE με use και only.
- Μπορώ να περάσω 1D array ως x(:) και να χρησιμοποιήσω size(x).
- Ξέρω πότε χρειάζεται allocatable και τη βασική ακολουθία allocate - χρήση - deallocate.